

两条路，一个终点：Context Graph

Agent 演进与 Memory 演进的殊途同归

WU Zihan

February 9, 2026

报告结构

总论与核心论点

Context Graph 概念框架

路线一：Agent 演进

路线二：Memory 演进

两条路线的交汇

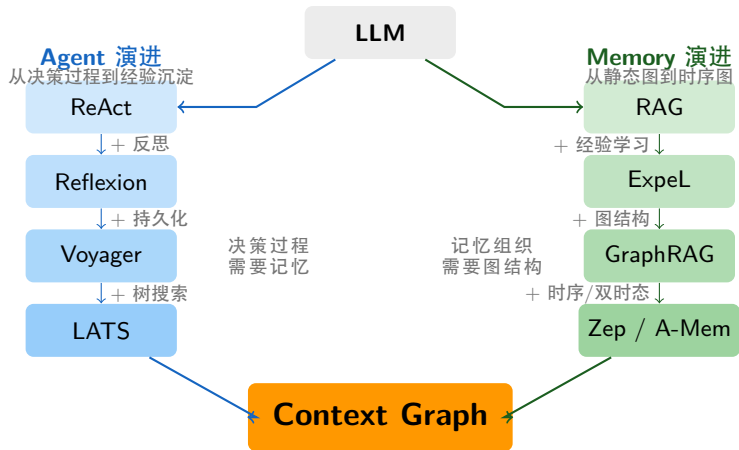
研究方向：Coding Agent

SWE-agent 轨迹实验分析

开放问题

总结与展望

技术路线图



为什么两条路都走向 Context Graph?

Agent 路线的困境

Reflexion: 反思存在列表中, 无法关联

LATS: 树搜索只覆盖单次任务

Voyager: 技能库是扁平的代码片段

Generative Agents: Memory Stream 线性增长

共同瓶颈: Agent 越强大,
产生的经验越多, 越需要结构化组织

Memory 路线的困境

RAG: 语义匹配丢失逻辑关联

GraphRAG: 静态文档, 无法记录决策

ExpeL: 洞察是列表, 缺乏图结构

A-Mem: 笔记网络缺少时序

共同瓶颈: 记忆越丰富,
越需要图结构来表达关系和推理

殊途同归

Agent 演进发现需要结构化记忆 $\leftarrow$ **Context Graph** $\rightarrow$ **Memory** 演进发现需要图结构

两条路的终点是同一个: 图结构 + 决策痕迹 + 动态学习的统一记忆架构

从 RAG 到 Context Graph 的演进

传统 RAG

- ▶ 文本块嵌入 + 向量相似检索
- ▶ **优点**：语义匹配能力强，部署简单
- ▶ **问题**：
 - ▶ 多跳推理能力弱
 - ▶ 丢失实体间关系
 - ▶ 无法回答全局性问题

GraphRAG / Context Graph

- ▶ 结构化三元组 (S, P, O) + 图遍历
- ▶ **优点**：多跳推理、关系显式建模
- ▶ **挑战**：
 - ▶ 图构建成本高
 - ▶ 实体解析与消歧
 - ▶ 动态更新与一致性

演进逻辑

结合两者优势：图结构的关系表达 + 上下文信息的丰富性 + 动态适应能力

为什么用图？——五大优势

能力	问题示例	RAG vs Graph
关系建模	Action A 之后应该做什么？	RAG: 找相似文档 → 可能不相关 Graph: 出边直接指向下一步 → 精确
多跳推理	A 导致 B, B 又触发 C?	RAG: 无"路径"概念 Graph: 搜索最短/最优路径
经验对比	成功 vs 失败的关键差异？	RAG: 找类似案例, 但不知差异 Graph: 比较成功/失败路径, 找分叉点
动态更新	新经验如何融入？	Fine-tuning: 需重训, 可能遗忘 Graph: 增量更新节点/边
可解释性	Agent 为什么这样决定？	Neural: 黑盒, 无法解释 Graph: " 因为 A→B 成功率 95%"

Context Graph：定义与定位

核心定义 [1]

Context Graph 是由决策痕迹（decision traces）编织成的活体记录结构，将实体、时间和先例连接成可查询的网络，使 AI Agent 能够理解“为什么这个决策被允许发生”。

与传统 KG 的区别

维度	传统 KG	Context Graph
设计目标	人类专家查询	AI 模型消费
核心要素	实体 + 关系	实体 + 关系 + 决策事件
时间处理	无或单时态	双时态（有效时间 + 事务时间）
溯源机制	可选	核心组件
更新方式	批量导入	实时/增量更新

三个视角

学术视角：GraphRAG 算法创新 产业视角：决策痕迹捕获 **Agent** 视角：结构化经验记忆

⁰Gupta, J. and Garg, A., *AI's trillion-dollar opportunity: Context graphs*, 2025.

GraphRAG vs Context Graph

核心区分

GraphRAG 是检索技术，**Context Graph** 是架构愿景。两者共享”图”结构，但解决不同层次的问题

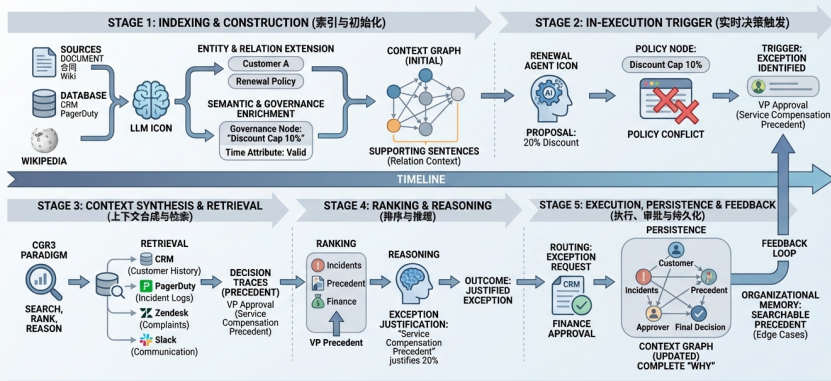
维度	GraphRAG	Context Graph
定位	检索技术	架构愿景
目标	改进 LLM 检索质量	捕获决策上下文
核心数据	静态文档语料	跨系统实时状态 + 决策事件
时态处理	静态快照	双时态（有效时间 + 事务时间）
溯源能力	引用来源文档	追溯完整决策推理路径
成熟度	2024-2025 学术验证充分	需要架构层面重新设计

类比

GraphRAG 之于 Context Graph $\approx$ TCP/IP 之于互联网：关键技术组件，完整实现需要更多系统工程

Context Graph 示例

CONTEXT GRAPH: ENTERPRISE CONTRACT RENEWAL PROCESS FLOW (BUILD TO APPLICATION) & DECISION TRACES



SUMMARY: Context Graph acts as **ORGANIZATIONAL MEMORY**, capturing reasoning logic (the "WHY") as First-Class Citizen Data for future AI agents.

Figure: Context Graph 如何组织决策过程与实体关系

Context Graph 节点与边类型

三种节点类型

类型 1: Action Node (动作节点)

- ▶ action_type: read_file | write_file | run_test
- ▶ parameters: {file: "auth.py", ...}
- ▶ result: success | failure | partial
- ▶ cost: token 消耗

类型 2: State Node (状态节点)

- ▶ description: "发现 bug 在 line 42"
- ▶ context: 相关文件、变量
- ▶ is_terminal: 是否终止状态

类型 3: Insight Node (洞察节点)

- ▶ insight: "Django migration 问题需先检查 models"
- ▶ confidence: 0.85
- ▶ source_tasks: [task_1, task_2, ...]

Action_1, Action_2, ... → Insight

四种边类型

类型 1: NEXT (时序/因果)

- ▶ Action_A $\xrightarrow{\text{NEXT}}$ Action_B
- ▶ 属性: success_rate, avg_reward, count

类型 2: CAUSES (因果)

- ▶ Action_X $\xrightarrow{\text{CAUSES}}$ Failure_Y
- ▶ "修改了 config 导致测试失败"

类型 3: SIMILAR_TO (相似)

- ▶ Task_1 $\xrightarrow{\text{SIMILAR_TO}}$ Task_2
- ▶ 用于跨任务经验迁移和模式复用

类型 4: ABSTRACTS (抽象)

- ▶ 多个具体经验归纳为通用规则

Agent 演进史

系统	核心创新	记忆形式	局限
ReAct (2023)	思考 + 行动交替	无持久记忆	每次从零开始
Reflexion [2]	语言化自我反思	反思列表（滑动窗口）	经验无法跨任务
Gen. Agents [3]	记忆流 + 反思 + 计划	Memory Stream	线性增长，无图结构
Voyager [4]	自动课程 + 技能库	代码技能库	扁平列表，无关联
LATS [5]	MCTS 树搜索 + 价值函数	搜索树节点	树不持久化

演进趋势：Agent 越强，越需要结构化记忆

ReAct（无记忆）→ Reflexion（反思列表）→ Voyager（扁平技能库）→ LATS（搜索树）→ **Context Graph**（结构化知识图）

⁰Shinn, N. et al., “Reflexion: Language Agents with Verbal Reinforcement Learning”, 2023.
⁰Park, J. S. et al., “Generative Agents: Interactive Simulacra of Human Behavior”, 2023.
⁰Wang, G. et al., “Voyager: An Open-Ended Embodied Agent with Large Language Models”, 2024.
⁰Zhou, A. et al., “Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models”, 2024.

Agent 演进：Reflexion & LATS

Reflexion [2]

语言化自我反思 (NeurIPS 2023)

- ▶ Actor 执行 → Evaluator 评估 → Self-Reflection 反思
- ▶ 反思存入 mem (长期记忆)
- ▶ AlfWorld +22%, HumanEval +11%

局限：滑动窗口 (1-3 条)，
反思之间无关联，无法跨任务

Shinn, N. et al., "Reflexion: Language Agents with Verbal Reinforcement Learning", 2023.

LATS [5]

MCTS 统一推理行动规划 (ICML 2024)

- ▶ 每步采样 n 个候选动作
- ▶ UCT 平衡探索与利用
- ▶ LLM 自评估 + Self-Reflection
- ▶ HumanEval **92.7%** (GPT-4 SOTA)

局限：搜索树不持久化，
每次任务从零重建

Zhou, A. et al., "Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models", 2024.

Context Graph = 持久化的搜索树 + 跨任务经验迁移

LATS 的树结构 (节点 = 状态, 边 = 动作, 权重 = 价值) 本质上就是 Context Graph 的雏形

Agent 演进: Voyager & Generative Agents

Voyager [4]

开放世界终身学习 **Agent**

- ▶ 自动课程生成（由易到难）
- ▶ GPT-4 生成可执行代码
- ▶ 技能存为代码 + 描述，语义检索复用
- ▶ 独特物品 3.3×、里程碑速度 15.3×

局限：技能库是扁平列表，
靠描述匹配检索，
无法表达技能间的依赖关系

Wang, G. et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models", 2024.

Generative Agents [3]

可信人类行为模拟 (UIST 2023)

- ▶ **Memory Stream**: 完整经历记录
- ▶ **Retrieval**: recency + importance + relevance
- ▶ **Reflection**: 高层推断生成
- ▶ **Planning**: 递归分解日程

局限：Memory Stream 线性增长，
反思与观察混在同一列表

Park, J. S. et al., "Generative Agents: Interactive Simulacra of Human Behavior", 2023.

Agent → Context Graph 的必然

Voyager 的技能库需要图结构表达依赖；Gen. Agents 的 Memory Stream 需要知识图谱组织实体关系。

Memory 演进史

系统	核心创新	记忆结构	局限
RAG (2020)	向量检索增强	文本块列表	丢失关系
ExpeL [6]	经验学习 + 洞察提取	洞察列表 + 轨迹	列表存储
GraphRAG [7]	社区检测 + 分层摘要	实体图 + 社区	静态快照
CGR ³ [8]	上下文增强 KG	扩展三元组	无 Agent 交互
Zep [9]	时序知识图谱	Episode+Entity+Community	静态时态，最接近 CG
A-Mem [10]	Zettelkasten 自组织	动态链接笔记	缺少时序

演进趋势：Memory 越高级，越趋向图结构

文本块 (RAG) → 经验列表 (ExpeL) → 实体图 (GraphRAG) → 时序图 (Zep) → **Context Graph**

⁰Zhao, A. et al., "ExpeL: LLM Agents Are Experiential Learners", 2024.

⁰Microsoft Research, *GraphRAG: Unlocking LLM discovery on narrative private data*, 2024.

⁰Xu, C. et al., "Context Graph", 2024.

⁰Rasmussen, P. et al., "Zep: A Temporal Knowledge Graph Architecture for Agent Memory", 2025.

⁰Xu, W. et al., "A-Mem: Agentic Memory for LLM Agents", 2025.

图增强检索：Microsoft GraphRAG [7]

核心问题

传统 RAG 无法回答全局性问题：
”这个语料库在讨论什么主题？”

方法流程

1. LLM 提取实体和关系
2. Leiden 算法社区检测
3. 自底向上生成分层社区摘要
4. 检索时结合局部 + 全局

实证结果 [11]

多跳推理和全局摘要 显著优于向量 RAG
但简单事实检索可能不如传统 RAG

Xiang, Y. et al., “When to use Graphs in RAG: A Comprehensive Analysis for Graph Retrieval-Augmented Generation”, 2025.

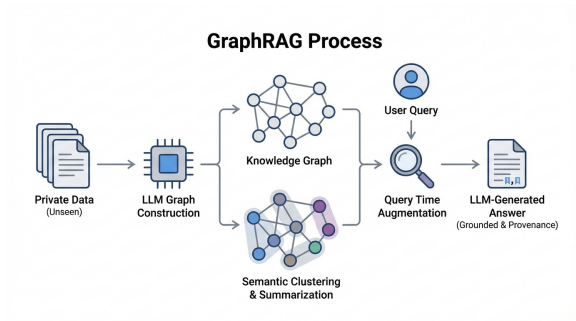


Figure: GraphRAG 分层索引架构

图增强检索: CGR³ [8]

核心问题

扩展三元组 KG, 融入时空、来源等上下文信息

CGR³ 范式

1. **Retrieve**: 候选实体和上下文
2. **Rank**: 基于上下文排序
3. **Reason**: 判断信息是否充足

关键发现

上下文信息对 KG 补全和问答任务至关重要

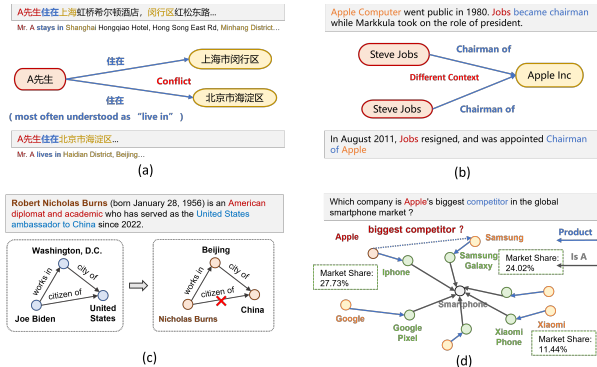


Figure: 传统三元组 KG 的四个局限性

- (a) 上下文丢失导致矛盾 (b) 无法区分相同关系的不同事实
(c) 规则推理忽略上下文 (d) 难以回答超出三元组范围的问题

GraphRAG 家族: RAPTOR、ToG、HippoRAG

RAPTOR [12]

递归抽象处理

- ▶ 文本块聚类生成层次化摘要树
- ▶ 自底向上递归构建多粒度索引
- ▶ QuALITY 基准提升 20%

ICLR 2024, Stanford

Sarathi, P. et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval", 2024.

Think-on-Graph [13]

LLM $\otimes$ KG 范式

- ▶ LLM 作为代理探索 KG
- ▶ 束搜索交互式推理
- ▶ 9 个 KGQA 中 6 个 SOTA

ICLR 2024, IDEA Research

Sun, J. et al., "Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph", 2024.

HippoRAG [14]

模拟海马体联想记忆

- ▶ LLM + KG + PPR
- ▶ Personalized PageRank 遍历图
- ▶ 多跳 QA 最高提升 20%

NeurIPS 2024, Ohio State

Gutiérrez, B. J. et al., "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models", 2024.

共同趋势

图结构 + LLM 推理能力的结合, 各有侧重: 层次摘要、交互探索、联想记忆

Agent Memory: ExpeL [6]

核心问题

LLM Agent 如何在不更新参数的情况下从经验中学习？

三阶段方法

1. 经验收集：Agent 与环境交互，存储成功/失败轨迹
2. 洞察提取：用 LLM 从轨迹中提取高层规则
3. 洞察驱动推理：新任务时检索相关洞察和轨迹指导决策

关键发现 (AAAI 2024 Oral)

- ▶ HotpotQA：洞察更重要 (36% vs 31%)
- ▶ ALFWorld：轨迹回忆更重要 (50% vs 55%)
- ▶ 两种记忆互补：不同任务类型需要不同侧重

对 CG 的启示

两种记忆形式：

- ▶ 具体轨迹 (episodic)
- ▶ 抽象洞察 (semantic)

Context Graph 需要同时支持这两种形式

Agent Memory: A-Mem [10]

核心问题

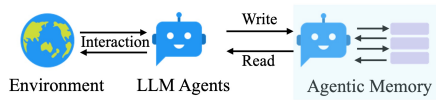
现有记忆系统结构固定，缺乏动态自组织能力

Zettelkasten 方法

1. 每条记忆生成上下文描述、关键词、标签
2. 动态链接：分析仓库建立语义连接
3. 新记忆触发旧记忆属性更新
4. 自主决定存储、链接或遗忘

实验结果 (NeurIPS 2025)

复杂推理性能翻倍，Token 消耗降低 85–93%，6 个模型均优于 SOTA



启示

Agentic Memory: 记忆系统本身具有“代理性”，主动组织、链接、更新，而非被动存储-检索

Agent Memory: 可训练图记忆 [15]

核心问题

如何结合隐式记忆（可适应但不可解释）和显式记忆（可解释但难优化）？

多层图架构

1. 底层：状态机抽象 Agent 轨迹
2. 中层：决策路径与战略元认知
3. 优化：基于 RL 的边权重调整

实验结果

- ▶ ALFWorld: +18% vs ReAct, +12% vs Reflexion
- ▶ WebShop: 成功率提升 15%，探索效率 $\times 2$
- ▶ 跨任务迁移：在新环境无需从零学习

对 Context Graph 的启示

图结构需要同时具备：

- ▶ 可训练：边权重可通过 RL 优化
- ▶ 可解释：人类可理解决策路径
- ▶ 可迁移：图结构跨任务复用

两条路线的方法全景对比

工作	路线	核心方法	记忆结构	持久化	图结构	发表
Reflexion	Agent	自我反思	反思列表	✗	✗	NeurIPS 23
Gen. Agents	Agent	记忆流 + 反思	Memory Stream	✓	✗	UIST 23
Voyager	Agent	技能库	代码列表	✓	✗	arXiv 23
LATS	Agent	MCTS 搜索	搜索树	✗	✓	ICML 24
ExpeL	Memory	经验学习	洞察列表	✓	✗	AAAI 24
GraphRAG	Memory	社区检测	实体图 + 社区	✓	✓	arXiv 24
A-Mem	Memory	Zettelkasten	动态链接	✓	✓	NeurIPS 25
Zep	Both	时序 KG	三层知识图谱	✓	✓	arXiv 25

关键观察

Zep/Graphiti 是两条路线的交汇点：既有 Agent 的动态交互能力，又有 Memory 的图结构组织。

但尚未应用于 **Coding Agent** 场景——这是研究空白。

两条路线的关键 Gap 与交汇机会

Agent 路线的 Gap

- ▶ Reflexion 的反思不持久化
- ▶ LATS 的搜索树不跨任务
- ▶ Voyager 技能库缺乏关系
- ▶ Gen. Agents 记忆流无图结构

⇒ 需要持久化、结构化的经验图

Memory 路线的 Gap

- ▶ GraphRAG 是静态的，无 Agent 交互
- ▶ ExpeL 洞察是列表，缺图结构
- ▶ A-Mem 缺少时序维度
- ▶ Zep 未应用于 Coding 场景

⇒ 需要动态、领域特定的记忆系统

交汇点：Context Graph for Coding

融合两条路线的优势：

- ▶ Agent 路线的动态交互能力
- ▶ Memory 路线的图结构组织
- ▶ 领域特定的 **Coding** 知识

核心创新：代码静态逻辑 (AST/Call Graph)

⊕ 动态决策轨迹 (Context Graph)

= 可生长的 Coding 记忆架构

FlexRule 警示 [16]

轨迹是“收据”，不是“食谱”

⇒ CG 需区分描述性 vs 规范性知识

Aghlara, A., *Context Graphs Design Flaws for AI Agents and Decision-Making*, 2024.

Coding Agent 任务全景

1. 代码生成 (Code Generation)

- ▶ 函数/类生成 (HumanEval, MBPP)
- ▶ 仓库级生成 (CodeAgentBench)
- ▶ 多轮交互式生成

2. Bug 修复 (Issue Resolution)

- ▶ GitHub Issue 修复 (**SWE-bench**)
- ▶ 自动程序修复 (APR)
- ▶ 回归 Bug 检测与修复

3. 漏洞检测 (Security)

- ▶ 安全漏洞发现 (CodeMender, Spark)
- ▶ Fuzz 测试与模糊化
- ▶ 形式化验证与静态分析

4. 测试生成 (Test Generation)

- ▶ 单元测试生成 (SWT-Bench)
- ▶ Fuzz 测试用例生成
- ▶ 回归测试与突变测试

5. 代码审查 (Code Review)

- ▶ PR 审查 (Qodo, CodeRabbit)
- ▶ 代码风格与规范检查
- ▶ 安全审计

6. 代码理解与导航

- ▶ 代码摘要与文档生成
- ▶ 依赖分析与影响评估
- ▶ 代码搜索与跨文件导航

主任务选择：Bug 修复 (SWE-bench)

为什么选 Bug 修复？

- ▶ 最主流的 Coding Agent 任务
- ▶ 成熟的 benchmark 和 leaderboard
- ▶ 有丰富的失败/成功轨迹数据

CG 能回答

- ▶ 这个 Bug 模式之前见过吗？
- ▶ 修复类似 Bug 的最优策略是？
- ▶ 哪些文件最可能需要修改？

当前痛点

1. 成功率

- ▶ 同类 Bug 反复出错
- ▶ CG：模式匹配加速定位

2. 效率

- ▶ 问题：token 消耗大
- ▶ CG：轨迹压缩，复用经验

3. 持续学习

- ▶ 每次任务从零开始
- ▶ CG：经验图跨任务复用

CG 辅助示例

问：这个项目怎么构建？

答：项目用 gradle 不用 maven

问：修复过程中哪步出错了？

答：图上路径清晰展示

问：下一步该做什么？

答：A→B 成功率 95%

方向 1: Bug 修复中的失败学习

核心问题

Coding Agent 在修复 Bug 时经常陷入循环：反复尝试相同失败方案

Context Graph 的作用

1. 记录每次修复的完整轨迹（成功 + 失败）
2. 自动提取 Bug 模式 → 修复策略规则
3. 建立 Bug 模式 → 修复策略的映射
4. 新任务时检索相似模式，避免重复犯错

相关研究

[17] 提出 AgentErrorTaxonomy, 系统分类 Agent 的失败模式（推理错误、工具误用、循环等）

关键研究问题

- ▶ 如何表示“失败原因”？
- ▶ 如何判断两个 Bug 是否相似？
- ▶ 如何避免过时策略误导 Agent？

Bug 修复 vs 漏洞检测：两个研究方向

维度	Bug 修复	漏洞检测
任务起点	已知 Issue 描述	未知，需主动发现
评估标准	Resolved Rate	检测率 + 误报率
知识依赖	项目代码逻辑	安全知识 (CWE/CVE)
CG 定位	失败学习与经验迁移	安全模式识别与知识图谱

为什么 Bug 修复优先？

- ▶ SWE-bench 基准成熟，便于验证效果
- ▶ 漏洞检测研究几乎空白，可作为后续

漏洞检测的机会

- ▶ 安全知识图谱 + Agent
- ▶ CWE/CVE 模式学习
- ▶ 自动化渗透测试

实验设计：验证 Context Graph 的价值

数据来源

SWE-bench Lite + SWE-agent (Llama-70B)

- ▶ 总轨迹：3,591 条
- ▶ 成功：342 条 (9.5%)
- ▶ 失败：3,249 条 (90.5%)

研究问题

1. Agent 失败的主要模式是什么？
2. 这些模式能否用图结构表达？
3. 成功经验能否迁移到失败案例？

三个实验

实验 1：循环检测与量化

→ 全量 3,591 条轨迹，验证循环假设

实验 2：失败轨迹图结构化

→ 100 条失败轨迹，验证 Schema 表达力

实验 3：跨轨迹模式匹配

→ 342 成功 vs 200 失败，验证经验迁移

核心假设

Context Graph 存储成功经验

⇒ 可避免 **Agent** 重复犯错

实验 1：循环检测与量化

循环定义

- ▶ 完全相同命令：如 edit 8:8 重复 N 次
- ▶ 相同动作类型：如连续 N 次 edit
- ▶ 相同动作 + 目标：如对同一文件反复搜索

阈值： $N \geq 3$ 次连续重复

关键发现 1：循环与失败强相关

	成功	失败
任意循环	28.7%	60.6%
严重循环	6.4%	24.3%

严重循环在失败案例中出现率是成功的 3.8 倍

关键发现 2：循环数量预测失败

阈值	成功	失败	比值
≥ 1 循环	28.7%	60.6%	2.1x
≥ 2 循环	6.7%	34.2%	5.1x
≥ 3 循环	2.0%	16.8%	8.4x

结论

- ▶ 循环检测可识别 **60%** 失败案例
- ▶ 有 3+ 循环 $\Rightarrow$ 失败概率 **8.4 倍**
- ▶ 整体恢复率 82.5%，但严重循环仅 68.5%

实验 2: 失败轨迹的图结构化

Schema 设计

节点类型

- ▶ ActionNode: 动作类型、命令、目标、是否重复
- ▶ CodeEntityNode: 文件/函数/类, 是否被修改
- ▶ ErrorPatternNode: 错误类型、触发动作
- ▶ FailurePatternNode: 失败模式、严重程度

边类型

- ▶ NEXT_ACTION: 动作序列
- ▶ TARGETS/MODIFIES: 动作 → 实体
- ▶ CAUSES: 动作 → 错误

转换结果 (100 条轨迹)

指标	平均值
动作节点数	31.1
实体节点数	4.5
错误节点数	15.6
失败模式数	2.9

失败类型分布

- ▶ loop_trap: **34%**
- ▶ error_accumulation: **34%**
- ▶ technical_error: 30%
- ▶ unknown: 2%

Schema 评估

- ✓ 可表达循环、技术错误
- ✗ 需增强: 任务理解、不完整重构

实验 3：跨轨迹模式匹配（核心创新）

核心思想

从成功轨迹提取修复策略子图，匹配失败轨迹的早期阶段，提供干预提示

策略提取

- ▶ 从 342 条成功轨迹提取 **278** 个策略模式
- ▶ 策略签名：
search→open→edit→test→submit
- ▶ 按任务类型分组：bug_fix, type_error, ...

匹配结果

- ▶ 尝试匹配：200 条失败轨迹
- ▶ 成功匹配：**190 条 (95%)**
- ▶ 平均分歧点：第 **8.4** 步

干预潜力分布

等级	数量	占比
High	61	32.1%
Medium	86	45.3%
Low	43	22.6%

77% 的匹配具有中高干预潜力

核心结论

95% 失败可匹配成功策略

⇒ Context Graph 存储经验可避免 **77%** 可预见失败

典型案例分析：四种失败模式

案例	步数	失败类型	关键错误
iterative/dvc-6933	226	循环陷阱 + 任务误解	功能请求当 Bug 修复，陷入超时循环 50+ 次
aio-lib/aioftp-113	6	技术知识错误	%s 在.strptime 无效，未验证即提交
mesa-1343	30	不完整重构	遗漏 @property 方法，edit 循环 10+ 次
networkx-3848	8	成功	路径直接：search→open→edit→submit

失败模式总结

1. 循环陷阱：相同命令重复执行
2. 任务误解：错误理解任务类型
3. 技术错误：API 知识缺陷
4. 不完整重构：遗漏关键代码

成功 vs 失败的关键差异

- ▶ 成功：路径直接，无探索迷失
- ▶ 失败：早期偏离 → 循环 → 越走越远
- ▶ 干预窗口：前 5-10 步最关键

案例深度剖析：循环陷阱的形成

案例：iterative/dvc-6933 (226 步，失败)

任务：为 DVC 添加 parallel coordinates plot 功能 (功能请求，非 Bug)

决策链分析

步骤 1-3：创建复现脚本

× 错误：这是功能请求，不是 Bug

步骤 4-10：遇到 pygit2 依赖错误

× 错误：尝试 pip install 解决环境问题

步骤 11-50：反复运行 dvc exp show

× 陷入超时循环 40+ 次

步骤 51-226：创建输出文件、重定向...

× 完全偏离正轨

核心洞察

Agent 从未尝试阅读代码
或理解如何实现该功能

完全陷入“复现 Bug”的
固定模式

CG 干预点

- ▶ 第 1 步：识别任务类型
- ▶ 第 4 步：检测环境问题
- ▶ 第 11 步：循环预警

实验总结：Context Graph 的价值验证

三个实验的核心发现

1. 循环是失败的强信号：检测到循环 $\Rightarrow$ 60% 概率失败，3+ 循环 $\Rightarrow$ 8.4x 失败率
2. 图结构可表达失败模式：Schema 能捕获 98% 的失败原因 (loop_trap, error_accumulation, technical_error)
3. 跨轨迹匹配可行：95% 失败可匹配成功策略，77% 具有中高干预潜力

对 Context Graph 的启示

应存储的内容

- ▶ 成功修复策略 (Action 序列)
- ▶ 失败模式 (循环、错误累积)
- ▶ 任务类型 $\rightarrow$ 策略映射

应提供的能力

- ▶ 实时循环检测与预警
- ▶ 策略匹配与下一步推荐
- ▶ 分歧点检测与早期干预

Schema 增强建议

新增 StrategyNode (修复策略模板)、MATCHES_STRATEGY 边、SUGGESTS_NEXT 边

Open Problems

● 图结构设计

Q1: 节点粒度

- ▶ 每个 action 一个节点?
- ▶ 每个 task 一个节点? 如何自动聚合?

Q2: 边的语义

- ▶ 因果/时序/相似如何区分?
- ▶ 边权重如何初始化和更新?

Q3: 动态更新

- ▶ 新经验融入如何避免图爆炸?
- ▶ 旧知识何时遗忘/压缩?

● 学习与优化

Q4: 检索效率

- ▶ 图规模增长后如何保持效率?
- ▶ 子图检索 vs 全图遍历?

Q5: 权重学习

- ▶ 用 RL? 监督学习?
- ▶ 如何避免对少量数据过拟合?

Q6: 探索 vs 利用

- ▶ 已知最优路径 vs 尝试新方案?
- ▶ UCB/epsilon-greedy 适用于图结构吗?

○ Coding 场景

Q7: Bug 表示

- ▶ 如何判断 Bug 相似性?
- ▶ 代码语义 vs 错误类型 vs 修复模式?

Q8: 代码变化

- ▶ 版本更新后图如何保持有效?
- ▶ API 变化如何传播到关联节点?

Q9: 评估指标

- ▶ 如何衡量 Context Graph 的价值?
- ▶ 超越 pass@k 的指标?

文献分类：两条路线 → 一个终点

Agent 演进路线

- ▶ **Reflexion** [2]: 语言化自我反思
- ▶ **LATS** [5]: MCTS 统一推理行动规划
- ▶ **Voyager** [4]: 开放世界技能库
- ▶ **Generative Agents** [3]: 记忆流 + 反思 + 计划

产业视角与评测

- ▶ Foundation Capital [1]
- ▶ FlexRule [16]: 批判视角
- ▶ GraphRAG-Bench [11]

Shinn, N. et al., "Reflexion: Language Agents with Verbal Reinforcement Learning", 2023.

Zhou, A. et al., "Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models", 2024.

Wang, G. et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models", 2024.

Park, J. S. et al., "Generative Agents: Interactive Simulacra of Human Behavior"

Memory 演进路线

- ▶ **GraphRAG** [7]: 社区检测 + 分层摘要
- ▶ **CGR³** [8]: 上下文增强三元组
- ▶ **ExpeL** [6]: 经验学习 + 洞察提取
- ▶ **A-Mem** [10]: Zettelkasten 自组织
- ▶ 可训练图记忆 [15]: 多层图 + RL

交汇点

- ▶ **Zep/Graphiti** [9]: 时序 KG Agent 记忆
- ▶ $\approx$ Context Graph 的生产级实现

Microsoft Research, *GraphRAG: Unlocking LLM discovery on narrative private data*, 2024.

Xu, C. et al., "Context Graph", 2024.

Zhao, A. et al., "ExpeL: LLM Agents Are Experiential Learners", 2024.

Xu, W. et al., "A-Mem: Agentic Memory for LLM Agents", 2025.

Xia, S. et al., "From Experience to Strategy: Empowering LLM Agents with Trainable Graph Memory", 2025.

Rasmussen, B. et al., "Zen: A Temporal Knowledge Graph Architecture for Agent

总结与下一步

核心论点

- ▶ **Agent** 演进 (Reflexion → Voyager → LATS): 越来越需要结构化记忆
- ▶ **Memory** 演进 (RAG → ExpeL → GraphRAG → Zep): 越来越趋向图结构
- ▶ 两条路的终点是同一个: **Context Graph** = 图结构 + 决策痕迹 + 动态学习

研究机会: Context Graph for Coding Agent

1. 失败学习图: 将 Reflexion 的反思 + LATS 的搜索树 → 持久化经验图
2. 技能依赖图: 将 Voyager 的技能库 → 有依赖关系的技能图谱
3. 代码知识图: 融合 Zep 的时序 KG + 代码结构 → Coding Context Graph

下一步

- ▶ 在 SWE-bench 上验证: Context Graph 能否减少 Coding Agent 的重复犯错?
- ▶ 基线对比: CG vs Reflexion vs ExpeL 在跨任务学习上的差异

参考文献 I

- [1] J. Gupta and A. Garg *AI's trillion-dollar opportunity: Context graphs*. URL: <https://foundationcapital.com/context-graphs-ais-trillion-dollar-opportunity/> 2025
- [2] N. Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning". *Advances in Neural Information Processing Systems* 2023
- [3] J. S. Park et al. "Generative Agents: Interactive Simulacra of Human Behavior". *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)* 2023
- [4] G. Wang et al. "Voyager: An Open-Ended Embodied Agent with Large Language Models". *arXiv preprint arXiv:2305.16291* 2024
- [5] A. Zhou et al. "Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models". *Proceedings of the 41st International Conference on Machine Learning (ICML)* 2024
- [6] A. Zhao et al. "ExpeL: LLM Agents Are Experiential Learners". *Proceedings of the AAAI Conference on Artificial Intelligence* 2024

参考文献 II

- [7] Microsoft Research *GraphRAG: Unlocking LLM discovery on narrative private data*. URL: <https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/> 2024
- [8] C. Xu et al. “Context Graph”. *arXiv preprint arXiv:2406.11160* 2024
- [9] P. Rasmussen et al. “Zep: A Temporal Knowledge Graph Architecture for Agent Memory”. *arXiv preprint arXiv:2501.13956* 2025
- [10] W. Xu et al. “A-Mem: Agentic Memory for LLM Agents”. *Advances in Neural Information Processing Systems* 2025
- [11] Y. Xiang et al. “When to use Graphs in RAG: A Comprehensive Analysis for Graph Retrieval-Augmented Generation”. *arXiv preprint arXiv:2506.05690* 2025
- [12] P. Sarthi et al. “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval”. *The Twelfth International Conference on Learning Representations* 2024
- [13] J. Sun et al. “Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph”. *The Twelfth International Conference on Learning Representations* 2024

参考文献 III

- [14] B. J. Gutiérrez et al. “HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models”. *Advances in Neural Information Processing Systems* 2024
- [15] S. Xia et al. “From Experience to Strategy: Empowering LLM Agents with Trainable Graph Memory”. *arXiv preprint arXiv:2511.07800* 2025
- [16] A. Aghlara *Context Graphs Design Flaws for AI Agents and Decision-Making*. URL: <https://www.flexrule.com/articles/context-graphs-design-flaws-for-ai-agents/> 2024
- [17] K. Zhu et al. “Where LLM Agents Fail and How They Can Learn From Failures”. *arXiv preprint arXiv:2509.25370* 2025